# ECG CSV BLE 転送 ― 不具合と対応策レポート

2026-10-28・エーアイシルク株式会社 岡野・対象：ecg_sender_ipad.py ／ ecg_viewer_iphone.py（r1〜r17、2026-10-24〜25 開発）・作成支援：Claude Code

**結論：**開発中に 16 件の不具合が出て、すべて解決しました。最終版 r17 で iPad → iPhone の転送と波形表示を実機で確認しています。原因の多くは「Pythonista 3.4 から iOS の CoreBluetooth を使うときの制約」で、スレッド・オブジェクトの寿命・呼び出し方の 3 点に集約されます。最終的には **BLE の処理を専用の作業スレッドに分け、CoreBluetooth を型を固定した関数で直接呼ぶ**構成で安定しました。

## 1. 不具合の全体像

区分ごとの件数と、どの版で解決したかを示します。実機でしか起きない問題が大半で、PC の自動テストでは再現しないため、実機の画面写真と動作記録を手がかりに 1 件ずつ切り分けました。




図 1　不具合 16 件の解決版と区分

## 2. 不具合一覧

| No. | 解決版 | 端末 | 症状 | 原因 | 対応策 |
|---|---|---|---|---|---|
| 1 | r1 | 両方 | Pythonista 標準の `cb` モジュールで書き込むとエラー（PY_SSIZE_T_CLEAN）。iOS 同士の接続もできない | `cb` はセントラル専用で、Pythonista 3.4 では書き込みが壊れている | `cb` を使わず、iOS の CoreBluetooth を直接呼ぶ。iPad はペリフェラル（CBPeripheralManager） |
| 2 | r1-5 | iPad | 起動直後に「NameError: free variable 's' referenced before assignment」。電波が出ず iPhone が見つけられない | 画面を出した直後にスクリプトが終わり、Pythonista が変数を片付けていた | 画面を閉じるまでスクリプトを終わらせない（最終的に `threading.Event().wait()`） |
| 3 | r1-5 | iPad | 「① CSVを選ぶ」を押しても反応しない | スクリプトを待たせる処理（wait_modal）の後ろにボタン処理が並び、実行されない | ファイル選択は専用スレッドで実行する |
| 4 | r6 | iPad | No.2 を直しても同じ NameError が続く | iOS から呼ばれる受け口（デリゲート）を入れ子の関数で作り、外側の変数を参照していた | デリゲートはモジュール直下の関数にし、動作中の送信側／受信側を登録表（_ACTIVE）から探して呼ぶ。テストで入れ子の関数を禁止 |

| No. | 解決版 | 端末 | 症状 | 原因 | 対応策 |
|---|---|---|---|---|---|
| 5 | r7 | iPhone | 「ArgumentError: 'int' object is not callable」 | タイマー予約（on_main_thread）に引数を渡していた | タイマーは `ui.delay` で、引数なしの関数だけを予約する |
| 6 | r7 | 両方 | 受信後、iPhone が iPad を探すモードに戻って接続を繰り返す。iPad は送信後に落ちる | 転送後の終わり方が決まっていなかった。名前による探索が、送信していない iPad にもつなぎにいく | 終了手順を定義：iPhone は応答後に切断して UUID で次を待つ。iPad は発信を止めて次の「送信」を待つ |
| 7 | r8 | iPad | 「② 送信」を押した直後に落ちる | ボタン用の別スレッドからタイマーを予約し、そのスレッドが先に終わっていた | タイマーはメインスレッドで予約する。テストで、メイン以外からの予約を検出 |
| 8 | r9 | iPhone | 名前で探す処理で ArgumentError。iPad に何度もつなぎ直す | サービスを指定しない全機器スキャンが Pythonista で失敗する | 名前による探索をやめ、サービス UUID だけで探す |
| 9 | r10 | iPad | 落ちるとコンソールが消え、どこで落ちたか分からない | 強制終了で画面の記録が失われる | 処理ごとに記録ファイルへ 1 行書き、すぐ保存（fsync）。最後の行が落ちる直前の処理になる |
| 10 | r11 | 両方 | 「'tuple' object is not callable」「'int' object is not iterable」などが不規則に出る | objc_util はスレッドに安全でない。呼び出しのたびに共有の型設定を書き換える | objc_util の呼び出しをメインスレッドだけに限定 |
| 11 | r12 | 両方 | メインスレッドに限定しても同じエラーが続く | iOS から次々呼ばれる処理の途中で、共有の objc_msgSend の型設定が壊れる | 呼び出しの型ごとに専用の objc_msgSend を ctypes で作り直接呼ぶ（`_send`）。objc_util はデリゲートのクラス作成だけに使う |
| 12 | r13 | 両方 | iPhone が接続すると iPad が落ちる。iPhone に「サービスがありません」 | 登録したサービス・特性を、登録直後に手放して（解放して）いた | サービス・特性は画面を閉じるまで保持。テストの模擬環境に参照数の確認を追加 |
| 13 | r16 | iPad | 記録ファイルが見当たらない。書き込みで PermissionError | 「ファイル」アプリから開いたスクリプトは、同じフォルダーに書き込めない | 書けないときは Pythonista の Documents に保存。スクリプトは「このiPad内 > Pythonista 3」に置いて実行する運用にした |
| 14 | r17 | iPad | iPhone がつながった瞬間に iPad が落ちる（Python の処理に入る前） | CoreBluetooth の操作と iOS からの通知の処理を、すべて画面と同じメインスレッドで行っていた | BLE の操作を専用の作業スレッドに移し、通知は値を渡すだけにする。画面は定期処理で更新し、別スレッドから ui.delay を使わない。落ちた位置を faulthandler で記録 |
| 15 | r17後 | iPad | BLE の準備完了とほぼ同時に「送信」を押すと、発信開始が 2 回行われる（実害なし） | 準備完了の処理が、送信待ちでない送信まで開始していた（スレッド間の競合） | 準備前に押された送信だけを、準備完了時に始める（wait_ble 印）。自動テストを追加 |
| 16 | r17後 | — | Google Drive に上げたファイルが 3 バイト短い | ソース中の見えない文字（BOM）がアップロード時に変換された | 見えない文字はエスケープ表記（`\ufeff`）で書く。アップロード後にサイズを照合 |

## 3. 主な不具合の詳細

### No.10・11　'tuple' object is not callable（呼び出しの型設定が壊れる）

**症状**　iPhone・iPad の両方で、特性の UUID を読むだけの呼び出しが不規則に失敗。エラーの文面も毎回少しずつ違った。

**原因**　objc_util は Objective-C を呼ぶたびに、共有の関数 objc_msgSend の引数・戻り値の型を書き換える。BLE は iOS から通知が次々と届くため、型を書き換えている最中に別の呼び出しが割り込み、型が壊れていた。メインスレッドに限定（r11）しても、通知の中で呼ぶ限り避けられなかった。

**対策**　呼び出しの型ごとに専用の関数を作り、型は一度決めたら変えない（`_send(obj, sel, sig, *args)`、例 `"B@@@"`）。objc_util はデリゲートのクラス作成と framework の読み込みだけに使う。

**確認**　ctypes のコールバックで objc_msgSend を模擬し、型ごとに正しく引数を渡せることを PC テストで確認。以後このエラーは出ていない。

### No.12　iPhone が接続すると iPad が落ちる（オブジェクトの解放）

**症状**　r12 で、iPhone がサービスを探しに来た瞬間に iPad が落ちる。iPhone 側には「サービスがありません」。

**原因**　直接呼び出しに変えたことで、オブジェクトの保持（retain）を自分で管理する必要が出た。登録したサービスと特性を登録直後に解放していたため、iOS が中身を見に来たときには解放済みだった。

**対策**　サービス・特性・接続中の相手は、閉じるか切断するまで保持する。テストの模擬環境に参照数を持たせ、「使用中に解放された」ことを検出できるようにした。

### No.14　iPhone 接続時に iPad が落ちる（スレッド構成）

**症状**　r16 で、「② 送信」→ 発信開始までは記録に残るが、iPhone がつながった瞬間に落ちる。記録の最後は「発信開始の通知」で、次の Python の処理に入っていない。

**調査**　診断プログラム（ble_diag_ipad.py）で、同じ手順を 1 段ずつ実行し、実際に iPhone の接続を待つ試験を行った。部品の作り方を 2 通り切り替えて比べる（A/B 試験）準備をしたが、1 回目の今の方式で落ちなかった。違いは「診断はスクリプト側のスレッドで BLE を操作し、通知は記録するだけ」という構成だけだった。

**対策**　送信プログラムを診断と同じ構成に作り替えた（図 2）。あわせて、別スレッドから画面の予約（ui.delay）をしない、落ちた位置を faulthandler で記録する、を加えた。

**確認**　実機で iPad → iPhone の転送と波形表示が成功。作業スレッドで送受信する自動テストも追加。

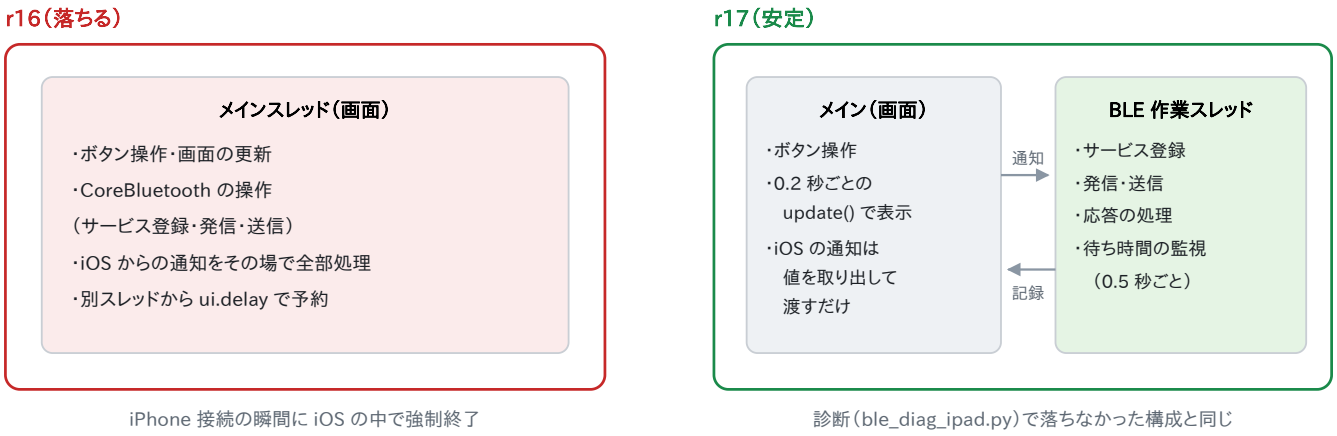



図 2　No.14 の対策：スレッド構成の変更

## 4. 調査の進め方

実機でしか起きず、落ちると手がかりが消える不具合が続いたため、次の順に「見える化」を強めました。

| 版 | 手段 | 分かるようになったこと |
|---|---|---|
| r1〜 | デリゲート内の例外を画面とコンソールに表示 | 見えずに止まっていたエラーの文面 |
| r6〜 | 起動時に版（BLE_VERSION）を表示 | どのファイルが動いているか（端末の取り違えを防止） |
| r10 | 処理ごとの動作記録をファイルへ即保存 | 落ちる直前の処理 |
| r14・r15 | 前回の記録を次の起動時に画面へ表示。Objective-C の呼び出しもすべて記録 | ファイルを探さなくても、画面写真 1 枚で前回の状況が分かる |
| r16 | 段階診断（ble_diag_ipad.py・ble_diag_sender_start.py） | BLE の準備 11〜12 段のどこで止まるか |
| r17 | iPhone 接続まで含む A/B 診断、faulthandler | 構成の違いによる差。iOS の中で落ちたときの Python の位置 |

並行して、PC 上で CoreBluetooth を模擬した自動テストを育て、直すたびに実機に渡す前に確かめました（最終 19 件）。実機で見つかった不具合のうち、模擬で再現できるもの（入れ子の関数、メイン以外からの予約、使用中の解放、スレッド間の競合など）は、テストで検出できるようにしています。

## 5. 再発防止策（Pythonista で BLE を扱うときの設計ルール）

1. **cb モジュールは使わない。**CoreBluetooth を直接呼ぶ。
2. **objc_util のメソッド呼び出しは使わない。**型を固定した専用の呼び出し関数を使い、objc_util はデリゲートのクラス作成だけにする。
3. **デリゲートはモジュール直下の関数にする。**入れ子の関数（クロージャ）にしない。
4. **iOS に渡したオブジェクトは自分で保持する。**サービス・特性・接続相手は、閉じるか切断するまで解放しない。
5. **BLE の操作は 1 本の作業スレッドに集める。**iOS からの通知では値を取り出して渡すだけにする。
6. **別スレッドから画面の予約（ui.delay）をしない。**画面は定期処理（update）で更新する。タイマーに引数を渡さない。
7. **スクリプトは画面を閉じるまで終わらせない。**待つ間は Python を動かさない（Event.wait）。
8. **探索はサービス UUID で行う。**名前での全機器スキャンはしない。
9. **転送の終わり方を両端末で決めておく。**応答 → 切断 → 次を待つ。
10. **落ちても追える仕組みを最初から入れる。**版の表示、即保存の動作記録、前回記録の表示、faulthandler。

11. **スクリプトは Pythonista 自身のフォルダーに置いて実行する。**「ファイル」アプリから直接開かない。

## 6. 残課題

| 項目 | 内容 | 対応案 |
|---|---|---|
| 通信の保護 | ペアリング・暗号化をしていない。近くで同じプログラムを動かす端末は受信できる | 製品データ・個人データを扱う前に、暗号化が必要な特性（権限）とペアリングを検討 |
| 動作記録の負荷 | 処理ごとにファイルへ保存しているため、その分遅い | 安定運用に入ったら、記録を止めるか間引く設定を追加 |
| 版の表示 | No.15 の修正後も、画面の版は「r17」のまま | 次に端末へ配る版で r18 に上げ、README・仕様書もそろえる |
| 転送速度 | 未測定 | 大きな CSV で実測し、仕様書に記載 |

## 7. 生成 AI 活用の観点からの所見

- 実機の画面写真やエラー文を渡すと、原因の推定 → 修正 → テスト追加 → 次の版の配布まで Claude Code が行った。人が行ったのは指示、実機での確認と結果の返送。
- No.14 のように原因が見えない問題では、「推測で直さず、比べる試験を作る」方針を AI 側から提案し、2 回の実機確認で解決した。
- 一方で、r11 → r12、r12 → r13 のように、1 つ直すと次の問題が出ることもあった。実機確認のたびに記録と診断を強化したことが、収束を早めた。
- 組込み開発への展開では、**実機での確認を人が素早く回せる体制**と、**落ちても手がかりが残る仕組み（記録・版表示・模擬テスト）を最初から入れること**が効果を左右する。